

Lab 12: Terraform Cloud Basics

Durée : 45 minutes

Cet atelier montre comment connecter Terraform Cloud à un système de gestion de code source (GitHub) et créer un espace de travail qui peut appliquer la configuration Terraform lorsque des modifications sont validées.

- Tâche 1 : Connecter GitHub à Terraform Cloud et créer un référentiel GitHub
- Tâche 2 : Créer des variables `public_key` et `private_key`
- Tâche 3 : Mettre un plan en file d'attente (Attention : cela échouera)
- Tâche 4 : modifier le code sur GitHub pour utiliser des variables au lieu d'un fichier
- Tâche 5 : Confirmer et appliquer le plan

Conditions préalables

Pour cet atelier, nous supposons que vous avez installé [Terraform](#) et que vous vous êtes [inscrit](#) pour un compte Terraform Cloud.

Tâche 1 : Connecter GitHub à Terraform Cloud et créer un référentiel GitHub

L'utilisation d'un référentiel GitHub nous permettra d'utiliser les meilleures pratiques de contrôle de source sur nos configurations d'infrastructure.

Le remplissage des variables dans Terraform Cloud donnera à Terraform Cloud nos informations d'identification AWS afin qu'il puisse exécuter Terraform en notre nom.

La connexion de Terraform Cloud à GitHub nous donnera un style de flux de travail d'intégration continue pour la gestion de l'infrastructure.

Étape 1.1 : Forkez le dépôt

Visitez ce référentiel GitHub et forkez-le pour en avoir une copie dans votre propre compte GitHub :

```
https://github.com/hashicorp/demo-terraform-101
```

Si vous le souhaitez, clonez le référentiel sur votre ordinateur local (si vous préférez modifier le code localement plutôt que dans le navigateur).

```
mkdir lab12; cd lab12
git clone https://github.com/$USER/demo-terraform-101.git
```

Nous allons travailler avec la branche `after-tfc`. Si vous choisissez de travailler localement, consultez cette branche:

```
git checkout -t origin/after-tfc
```

Étape 1.2 : connectez GitHub à Terraform Cloud et créez un espace de travail

Allez maintenant sur app.terraform.io

Nous allons connecter notre référentiel GitHub à Terraform Cloud. Si vous n'avez pas encore d'organisation dans Terraform Cloud, créez-en une. Si vous avez été invité dans une organisation Terraform Cloud existante, vous pouvez également y accéder.

Vous verrez une page vide où seront vos lieux de espaces de travail. Cliquez sur le bouton "New Workspace" en haut à droite.

Sur la "New Workspace page", sélectionnez GitHub -> GitHub.com pour continuer. Une nouvelle fenêtre devrait s'ouvrir vous demandant d'autoriser Terraform Cloud à votre compte GitHub. Si vous ne vous êtes pas connecté à GitHub récemment, vous devrez peut-être d'abord vous connecter.

Create a new Workspace

Workspaces allow you to organize infrastructure and collaborate on Terraform runs.

1 Connect to VCS

2 Choose a repository

3 Configure settings

Connect to a version control provider

Choose the version control provider that hosts your Terraform source code. If you're planning to [upload your configurations via the API](#), you can [skip this step](#).



New to Terraform? [Take a tutorial](#)

Cliquez sur le bouton vert Authorize . . . pour connecter Terraform Cloud à votre compte GitHub.

Terraform Cloud by **HashiCorp** would like
access to:



Your account (robin-norwood)

Verify your GitHub account



Resources

Determine what resources both you and Terraform Cloud can
access

Terraform Cloud has been installed on 1 account you have access to:
hashicorp.

[Learn more about Terraform Cloud](#)

Authorize Terraform Cloud by HashiCorp

Authorizing will redirect to
<https://app.terraform.io>

En cas de problème, essayer avec l'option **Github.com (custom)**

Ensuite, vous verrez une liste de vos référentiels GitHub. Choisissez le référentiel que vous avez créé lors de la première étape.

Create a new Workspace

Workspaces allow you to organize infrastructure and collaborate on Terraform runs.



Connect to VCS



Choose a repository



Configure settings

Choose a repository

Choose the repository that hosts your Terraform source code. We'll watch this for commits and pull requests.

 **robin-norwood** ▾ 1 repository

demo-terraform-101 >

Can't see your repository? Enter its ID below, e.g. `acme-corp/infrastructure` :



À la dernière étape, laissez le nom de l'espace de travail et les "Options avancées" inchangés, puis cliquez sur le bouton violet **Create workspace** pour créer l'espace de travail.

Il faudra quelques minutes à Terraform Cloud pour se connecter à votre référentiel GitHub. Une fois cette opération terminée, vous devriez voir une notification indiquant que votre configuration a été téléchargée avec succès.



Configuration uploaded successfully

Your configuration has been uploaded. Next, you probably want to configure variables (such as access keys or configuration values). If your configuration doesn't require variables, you can queue your first plan now.

[Configure variables](#)[Queue plan](#)

Étape 1.3 : Configurer les variables

Allez dans l'onglet "Variables". Sur la page des variables, vous verrez qu'il existe deux types de variables :

- Variables Terraform : elles sont introduites dans Terraform, de la même manière que `terraform.tfvars`
- Variables d'environnement : elles sont renseignées dans l'environnement d'exécution où Terraform s'exécute

Dans la section "Variables", cliquez sur "Add variable" et ajoutez des clés et des valeurs pour toutes les variables dans le fichier `variables.tf` du projet. Vous aurez besoin de votre `identity` qui est votre nom unique ainsi que `ami` qui devrait être `ami-0f29c8402f8cce65c`.

Tâche 2 : Créer des variables `public_key` et `private_key` dans Terraform Cloud

Étape 2.1 : Configurer les clés SSH

Terraform Cloud s'exécute dans un environnement avec des privilèges limités. Il n'a pas accès aux clés SSH de votre instance qui ont été lues à partir du stockage local lorsque ce code a été écrit à l'origine.

Commencez par copier le contenu des clés SSH publiques et privées de l'instance dans Terraform Cloud en tant que variables Terraform.

Voici une façon de les afficher dans votre terminal d'une manière qui peut être copiée dans votre presse-papiers local.

Utilisez `cat` avec chaque fichier dans le terminal, copiez-les dans le presse-papiers et créez des variables sur Terraform Cloud nommées `public_key` et `private_key`. Assurez-vous de cliquer sur **Save Variable** après avoir entré chaque clé et valeur.

```
cat ~/.ssh/id_rsa.pub
ssh-rsa AAAA...
```

Vous pouvez marquer des valeurs comme **sensitive**, ce qui les rend "en écriture seule" (elles ne seront pas affichées dans l'interface utilisateur après leur enregistrement). Assurez-vous de le faire au moins avec votre clé privée.

```
cat ~/.ssh/id_rsa
-----BEGIN RSA PRIVATE KEY-----
MII...
```

Vous DEVEZ appuyer sur **Save Variable**. Sinon, les variables ne seront pas enregistrées.

Étape 2.2 : Saisir les informations d'identification AWS

Il y a aussi une section pour les variables d'environnement. Nous les utiliserons pour stocker les informations d'identification AWS.

Ajoutez des variables d'environnement pour vos informations d'identification AWS. : -
`AWS_ACCESS_KEY_ID` - `AWS_SECRET_ACCESS_KEY` - `AWS_DEFAULT_REGION`

Assurez-vous de marquer `AWS_SECRET_ACCESS_KEY` comme "Sensible".

Cliquez sur le bouton "Save variable" pour chacun.

Tâche 3 : Mettre un plan en file d'attente

Pour cette tâche, vous mettrez en file d'attente terraform plan.

Étape 1 : Mettre un plan en file d'attente et lire le résultat

Cliquez sur le bouton Actions puis Start a new plan en haut à droite. On vous demandera une raison, que vous pouvez définir comme vous le souhaitez. Cela vous amènera à la page Détails de l'exécution Runs.

Faites défiler jusqu'à l'endroit où il affiche le plan. Vous verrez le même type de sortie que vous avez l'habitude de voir sur la ligne de commande.

Nous apporterons une autre modification à partir de GitHub avant d'exécuter ce plan, alors ne vous méfiez pas de l'erreur d'exécution .

Tâche 4 : Modifier le code sur GitHub pour mettre à niveau la version du provider AWS

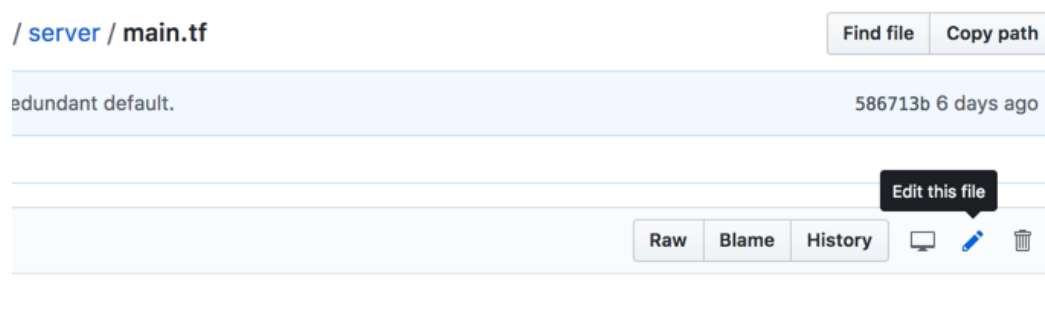
Modifiez le code sur GitHub pour mettre à niveau la version du provider AWS vers `>= 2.27.0`.

Vous ferez une demande d'extraction avec ces modifications et observerez l'état de la demande d'extraction sur GitHub.

Étape 4.1

Sur GitHub, recherchez le menu déroulant "Branch" et passez à la branche "after-tfc".

Accédez à `main.tf`. Trouvez l'icône du crayon. Cliquez pour éditer ce fichier directement dans le navigateur.

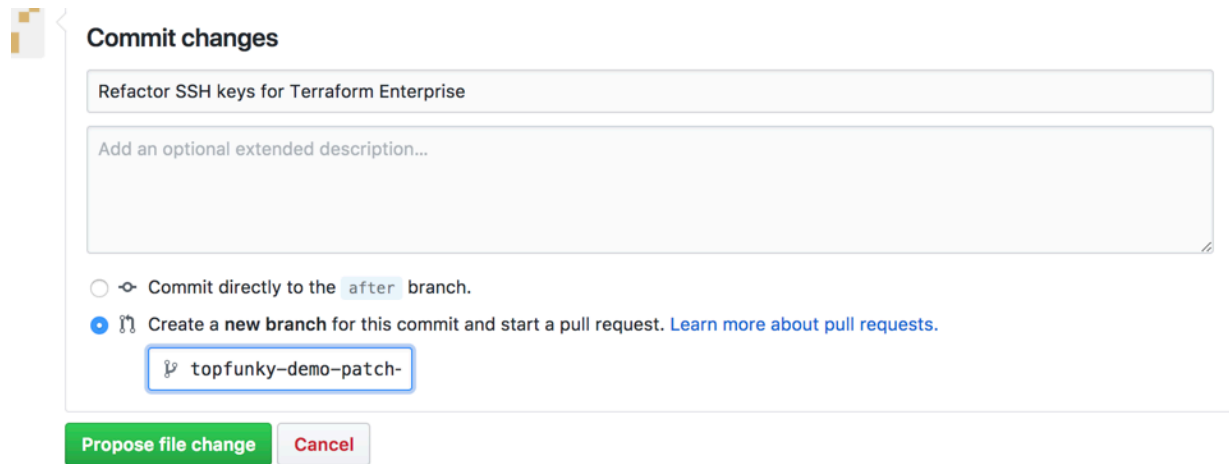


Modifiez le code pour qu'il corresponde aux lignes ci-dessous.

```
provider "aws" {  
  # MODIFY this line to look for 2.27.0 or greater
```

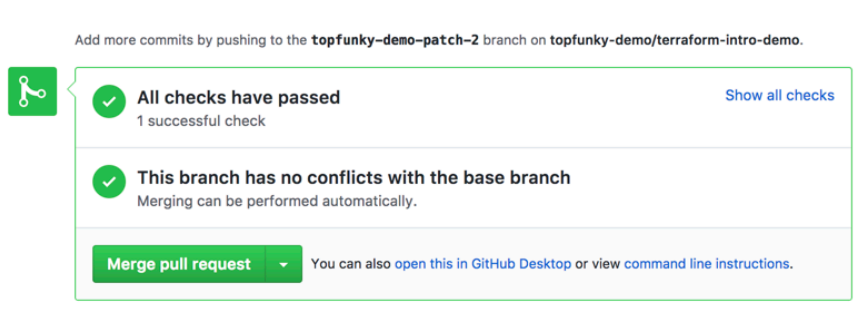
```
version = ">= 2.27.0"
}
```

Faites défiler vers le bas et sélectionnez l'option "Create a new branch and start a pull request."



Vous serez redirigé vers un écran pour créer une pull request. Cliquez sur le bouton vert **Propose file change**. La page sera pré-remplie avec votre message de validation. Cliquez sur "Create pull request."

Après quelques secondes, vous verrez que Terraform Cloud a vérifié le plan et qu'il a réussi.



Pour voir ce qui a été coché, cliquez sur "Show all verifications" et cliquez sur "Détails" à côté de la ligne indiquant "Le plan Terraform a des modifications".

Fusionnez la pull request à la branche `after-tfc` avec le gros bouton vert **Merge pull request**. Cliquez sur le bouton **Confirm merge**.

Tâche 5 : Confirmer et appliquer le plan

Étape 5.1 : Confirmer et "appliquer"

De retour sur Terraform Cloud, recherchez l'onglet **Current Run**. Cliquez dessus et vous verrez que le commit de fusion a déclenché un plan et qu'il doit être confirmé.

rom caseylang/caseylang-patch-1

! NEEDS CONFIRMATION

caseylang triggered from GitHub | Branch `after-tfe` | `b21b55a`

2 minutes ago

Après avoir confirmé le "plan", allez au bas de la page et cliquez sur `Confirm` and `apply`. Vous aurez la possibilité de laisser un commentaire (facultatif). Enfin, cliquez sur "Confirm plan".



Apply finished 8 minutes ago

Cela mettra en file d'attente un `terraform apply`.

Examinez la sortie de `apply` et recherchez l'adresse IP de la nouvelle instance. La sortie ressemble à ce que vous avez vu précédemment dans le terminal. Copiez l'adresse `public_ip` et collez-la dans votre navigateur. Vous verrez l'application Web en cours d'exécution.



Apply finished 8 minutes ago

Queued 11 minutes ago > **Started** 11 minutes ago > **Finished** 11 minutes ago

 View raw log

```
module.server.aws_instance.web (remote-exec): Private key: true
module.server.aws_instance.web (remote-exec): SSH Agent: false
module.server.aws_instance.web (remote-exec): Connected!
module.server.aws_instance.web: Creation complete after 27s (ID: i-0936d03b68874f0b6)
```

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

```
public_dns = [
  ec2-34-212-49-75.us-west-2.compute.amazonaws.com
]

public_ip = [
  34.212.49.75
]
```

Tâche 6 : Détruire

Pour nettoyer, détruisez l'infrastructure que vous venez de créer.

Étape 6.1 : Configurer la variable CONFIRM_DESTROY

Accédez à l'onglet "Settings" de l'espace de travail dans Terraform Cloud et faites défiler vers le bas. Notez les instructions sous « Delete workspace ». Nous voulons détruire l'infrastructure mais pas nécessairement l'espace de travail.

Vous devrez créer une variable d'environnement (pas une variable Terraform) nommée "CONFIRM_DESTROY" et la définir sur "1".

Allez dans l'onglet "Variables" et faites-le.

Confirmer la destruction

Confirmer la destruction

Cliquez sur "Save variable".

Étape 6.2 : Plan de destruction de la file d'attente

Retournez maintenant dans l'onglet "Settings". Faites défiler vers le bas et cliquez sur "Queue destroy plan". Notez les messages sous "Plan" qui indiquent qu'il va détruire plusieurs ressources.

Cliquez sur "Confirm and apply". Après quelques secondes, votre infrastructure sera détruite comme demandé.